A Tale of Five Decoders

David Ponarovsky

November 6, 2025

Introduction

Today:

- Recap and where we stood before the summer.
- Simulations results.
- Where we heading.

Map

Coming with $O(1)$-depth gates for 'good' codes is hard.

Map

Coming with $O(1)$-depth gates for 'good' codes is hard.

Is the (d, k)-Toric a MEM?

Map

Coming with $O(1)$-depth gates for 'good' codes is hard.

Is the (d, k)-Toric a MEM?

Dennis at el? [Den+02]

Map

Coming with $O(1)$-depth gates for 'good' codes is hard.

↓

Is the (d, k)-Toric a MEM?

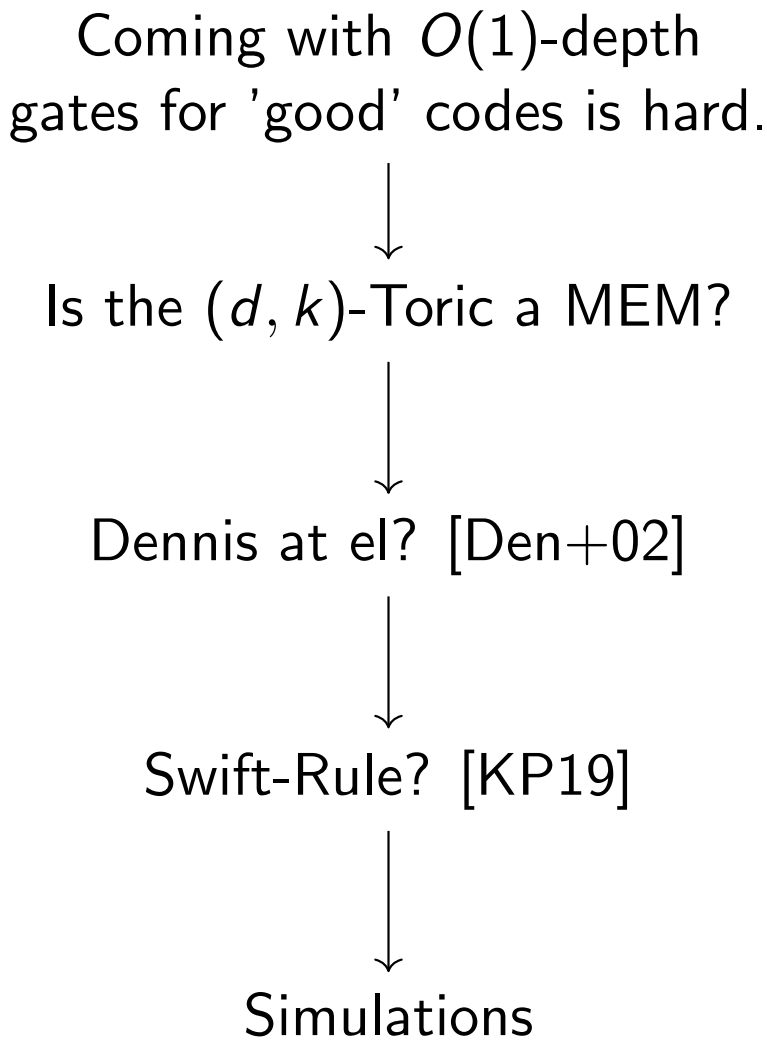
↓

Dennis at el? [Den+02]

↓

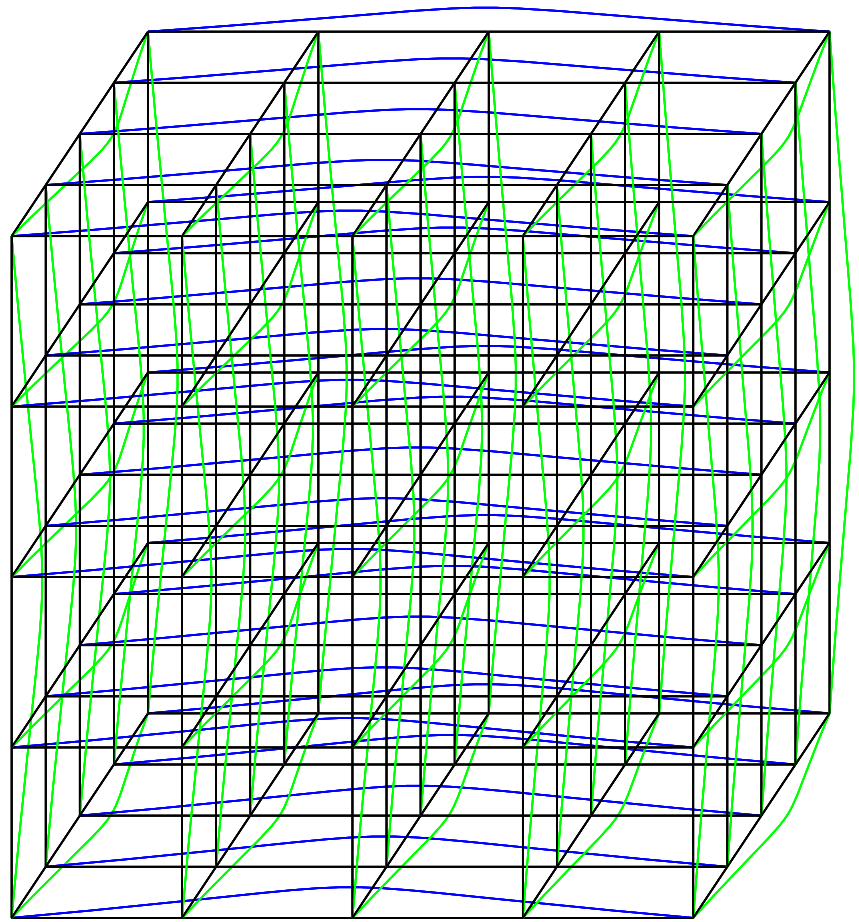
Swift-Rule? [KP19]

Map


Coming with $O(1)$-depth
gates for 'good' codes is hard.
Is the (d, k)-Toric a MEM?
Dennis at el? [Den+02]
Swift-Rule? [KP19]
Simulations

The Code

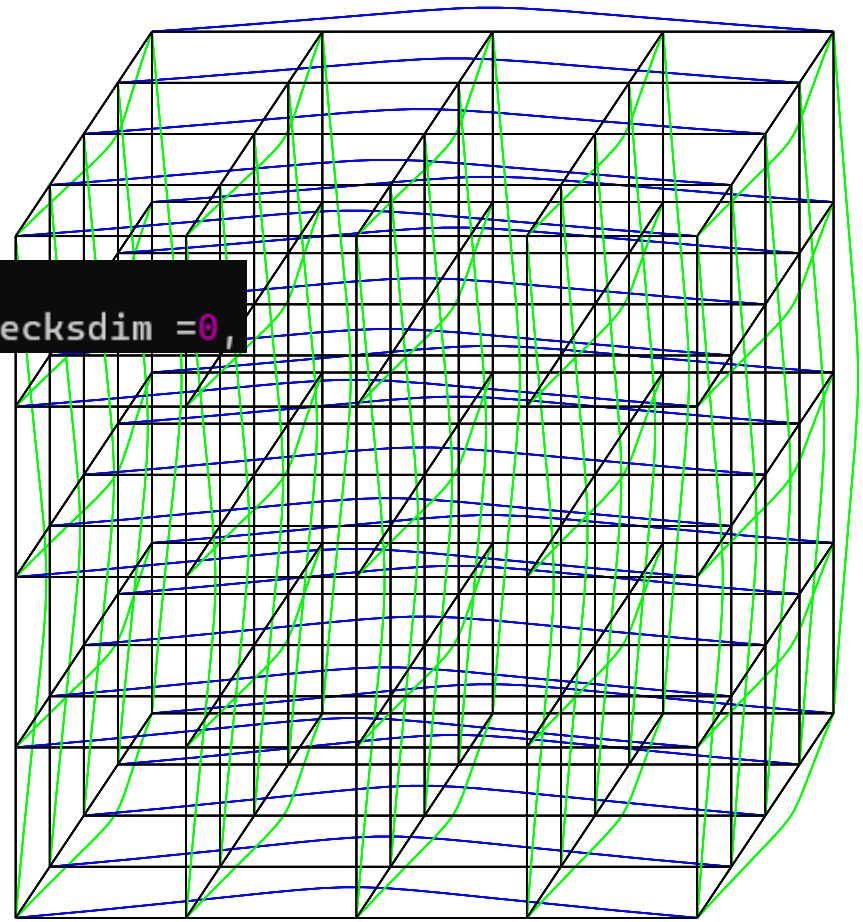
The simulations are over the 3D-Toric
where qubits set on faces
and checks on vertices.



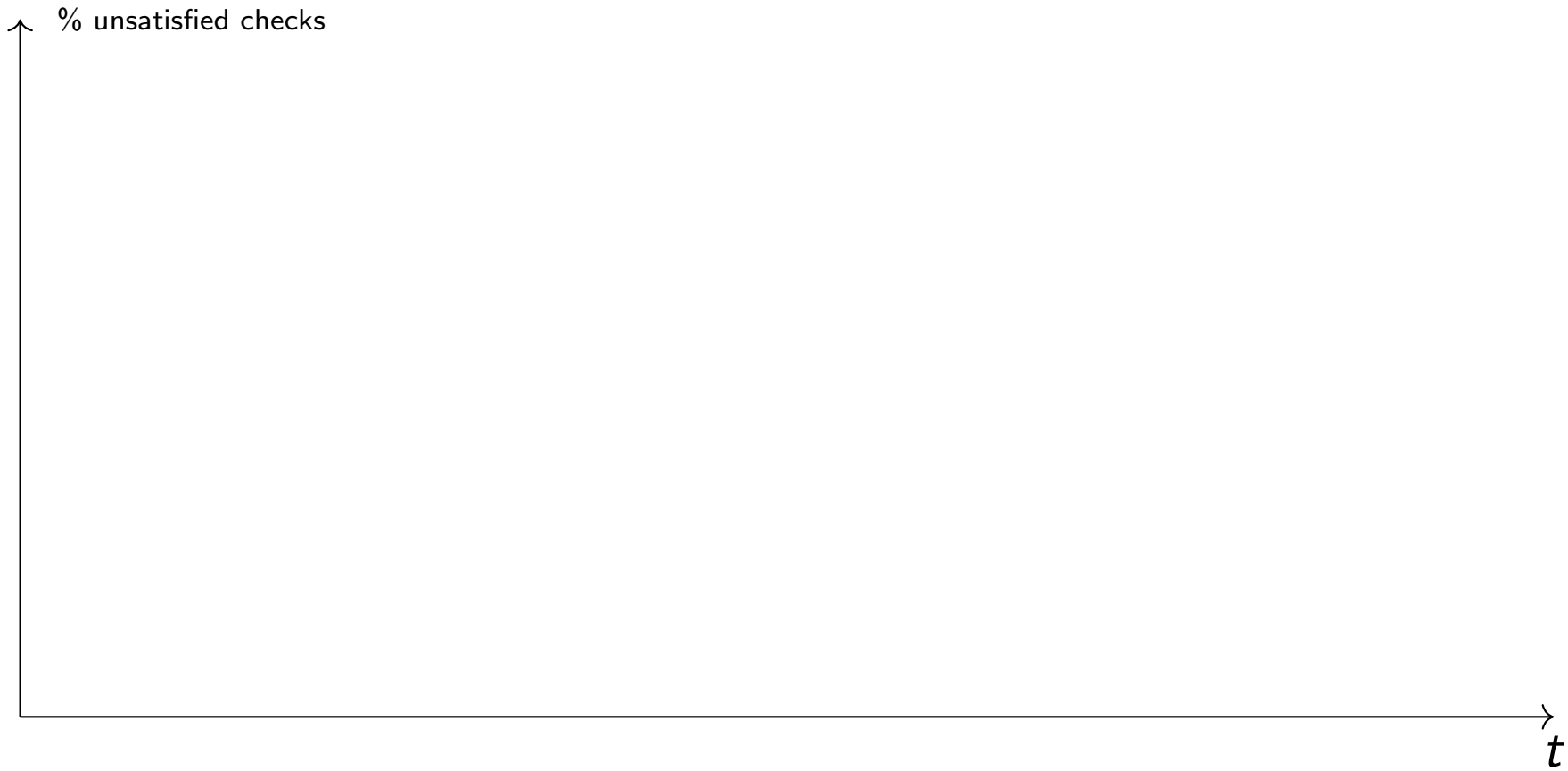
The Code

The simulations are over the 3D-Toric where qubits set on faces and checks on vertices.

```
class KDgrid:
    def __init__(self, n, k, celldim=2, checksdim =0,
```

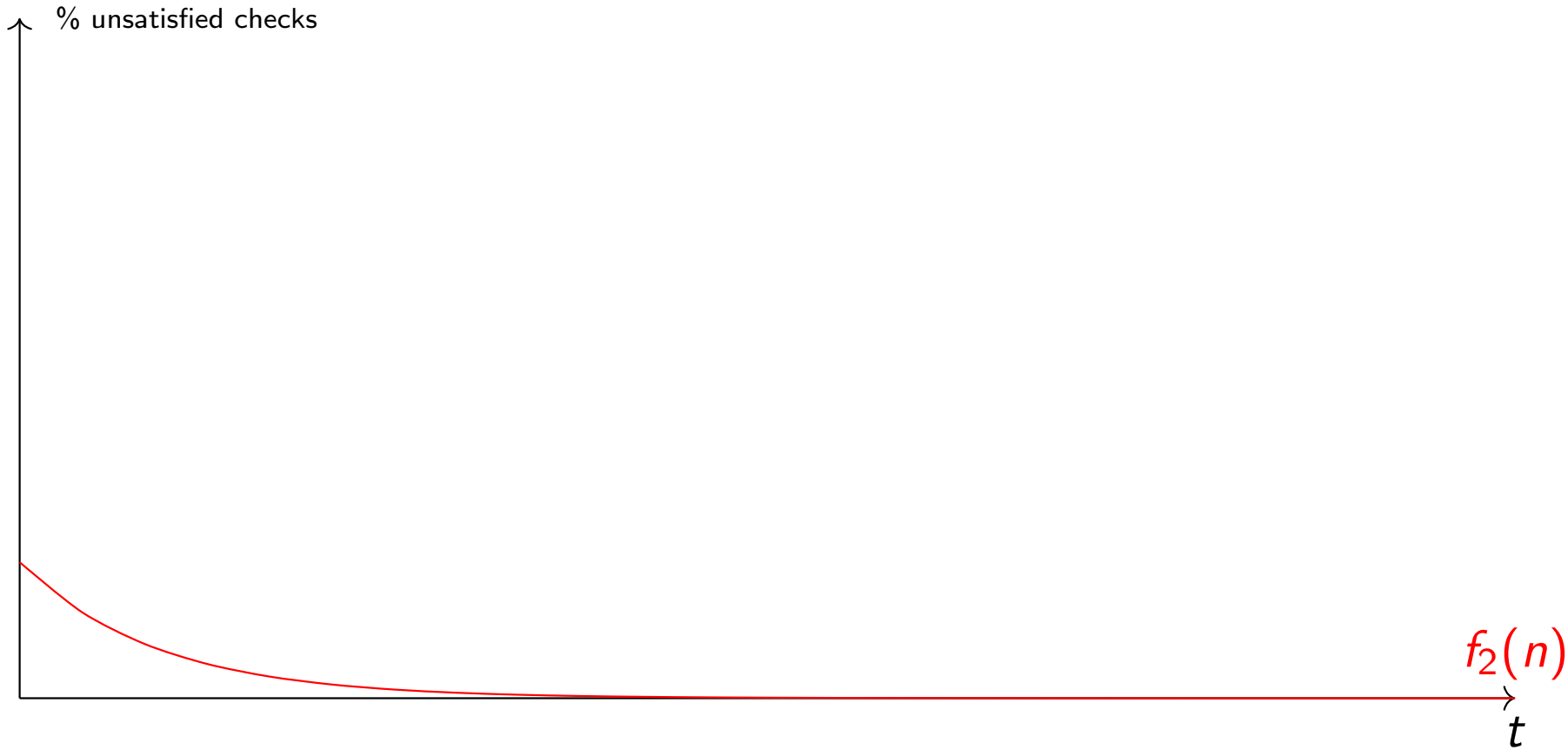


How to Read



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

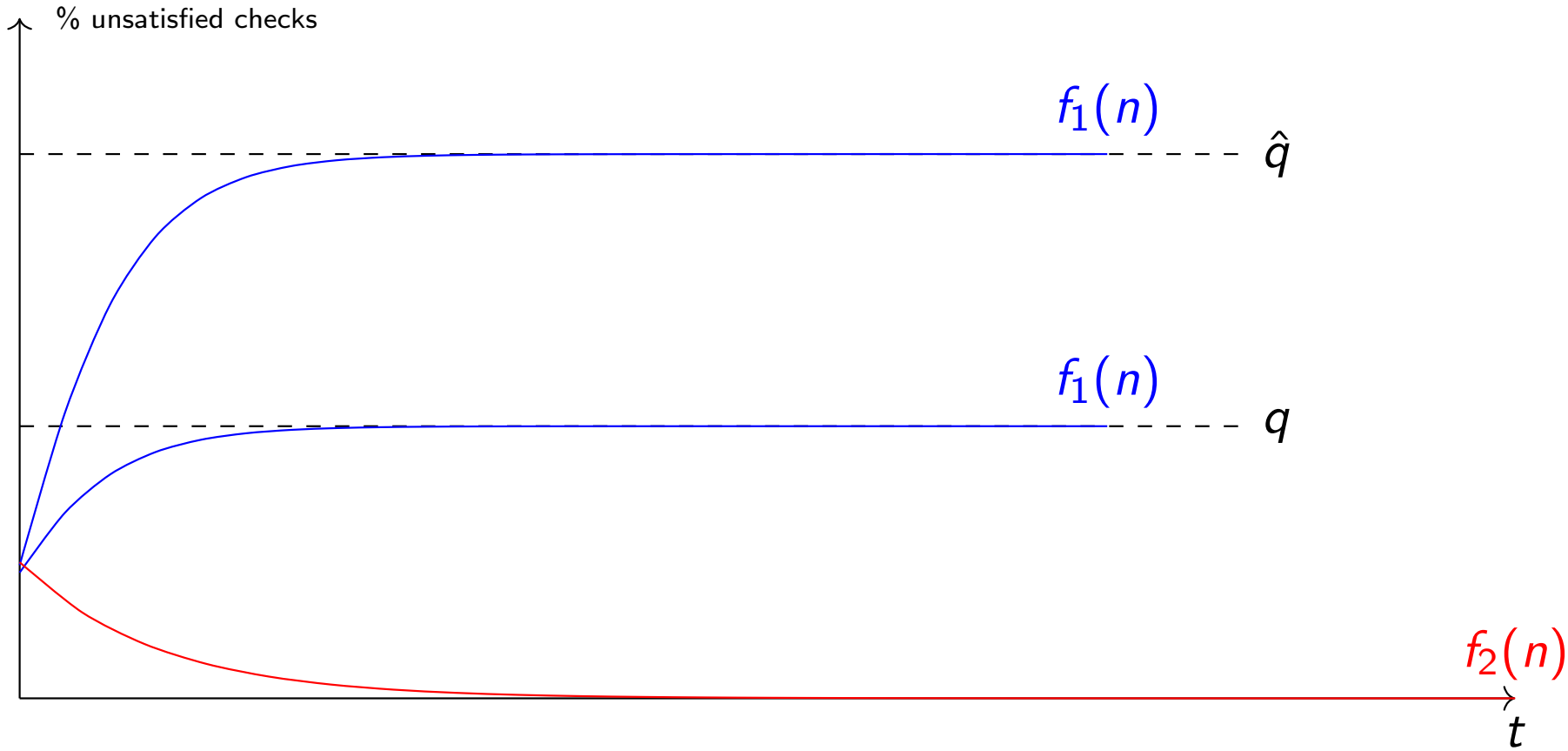
Ideal Result.



colors

In red, No accumulation of errors.

Ideal Result.

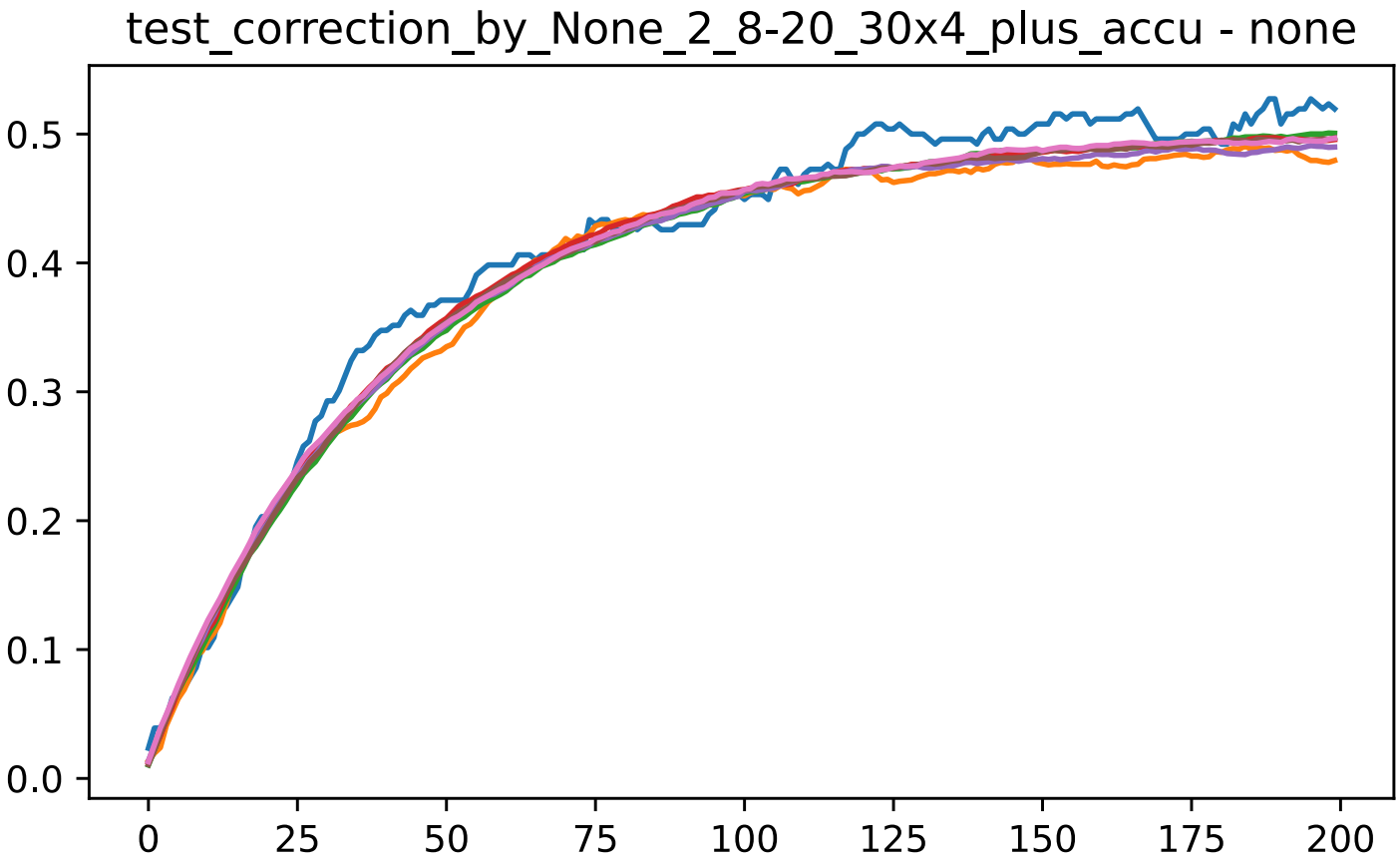


colors

In red, No accumulation of errors.

In blue, Decoding in the presence of noise.

Without Decoding



side-length :[8, 16, 30, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

Majority

Alg.

Each bit looks at its neighbour checks, if the majority of them is unsatisfied, then it flip itself.

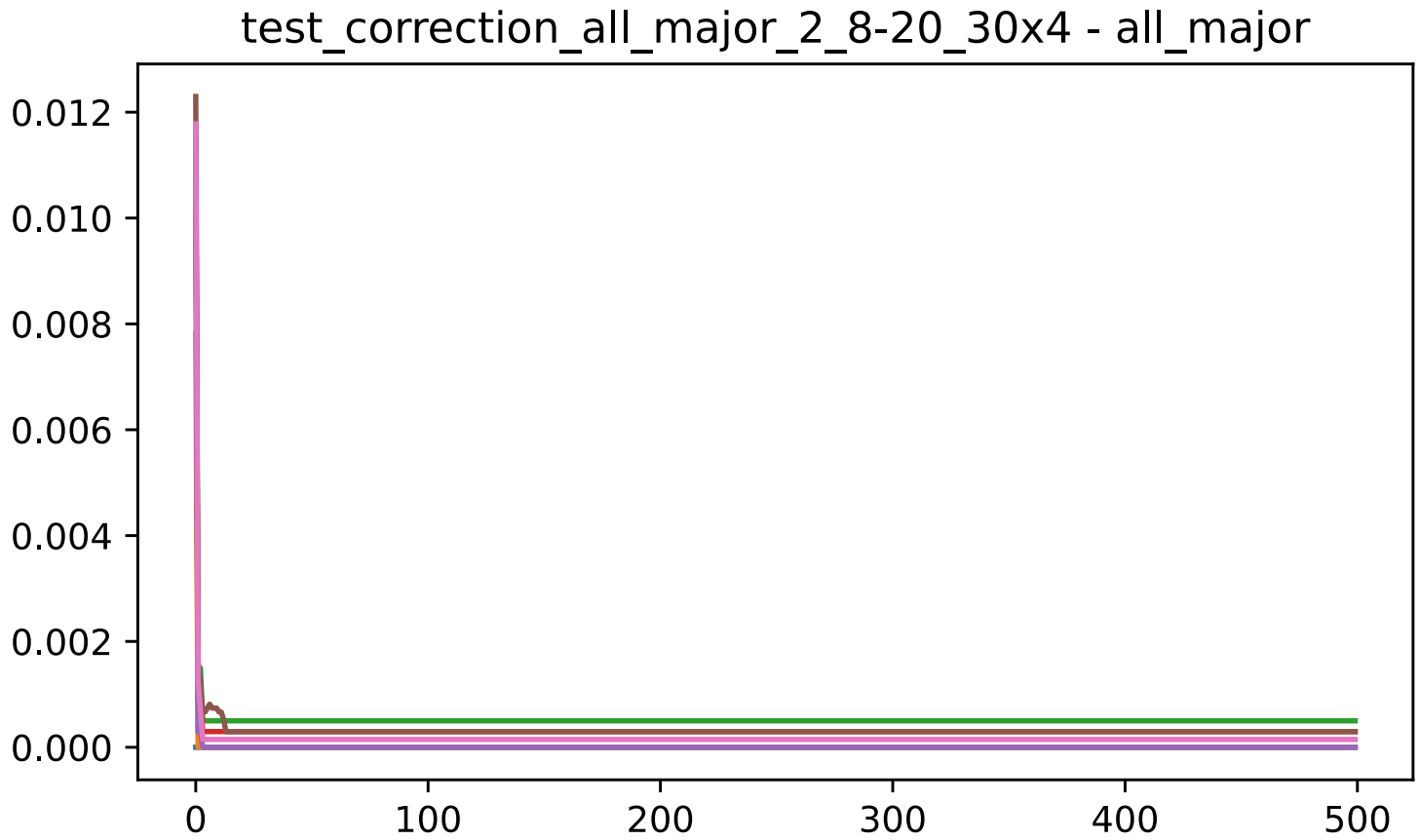
Justification.

Was suggested in Dennis.

Bottom line.

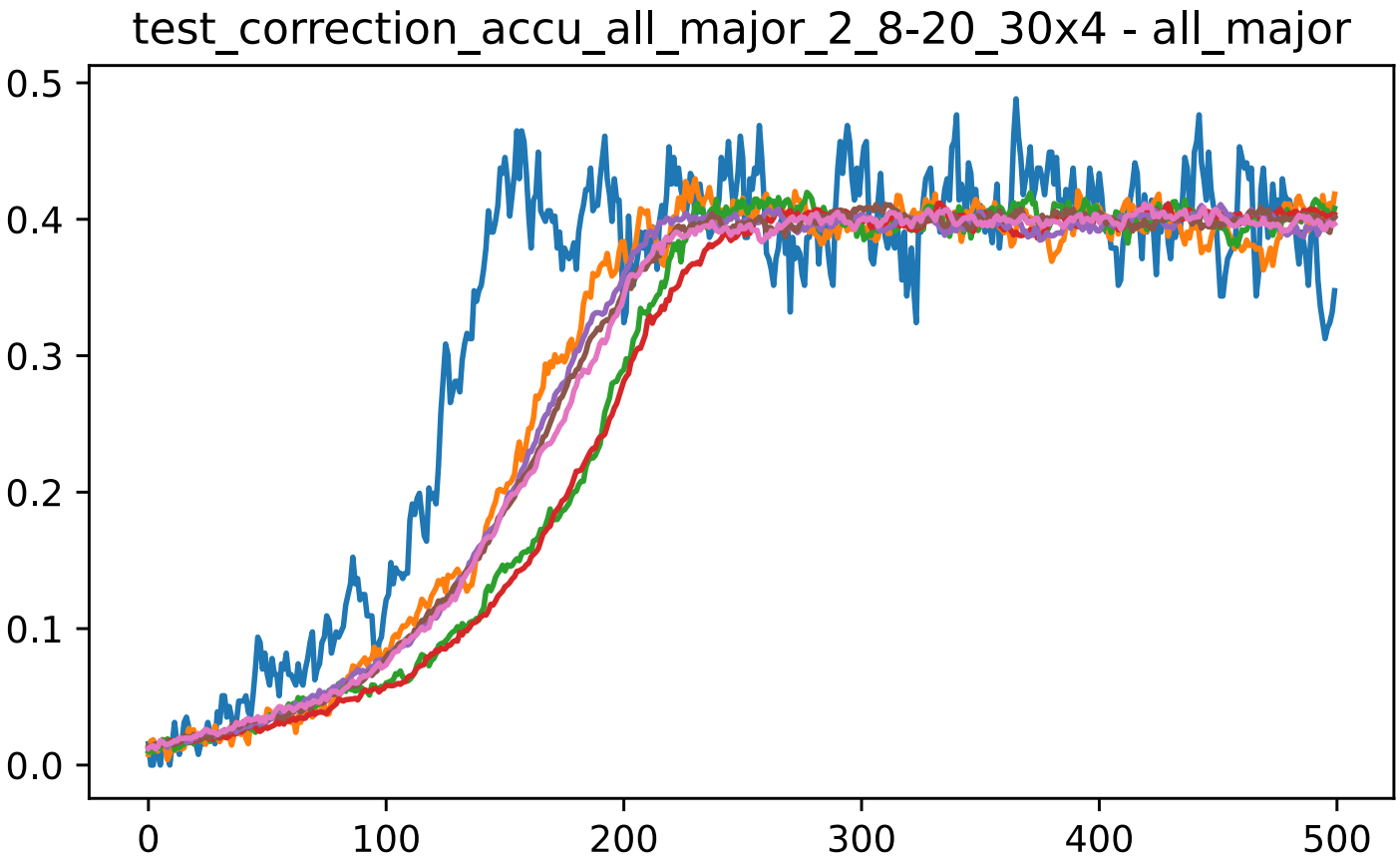
- Collisions.
- Gives better result when requiring $(\frac{1}{2} + \mu)$-majority.

Majority



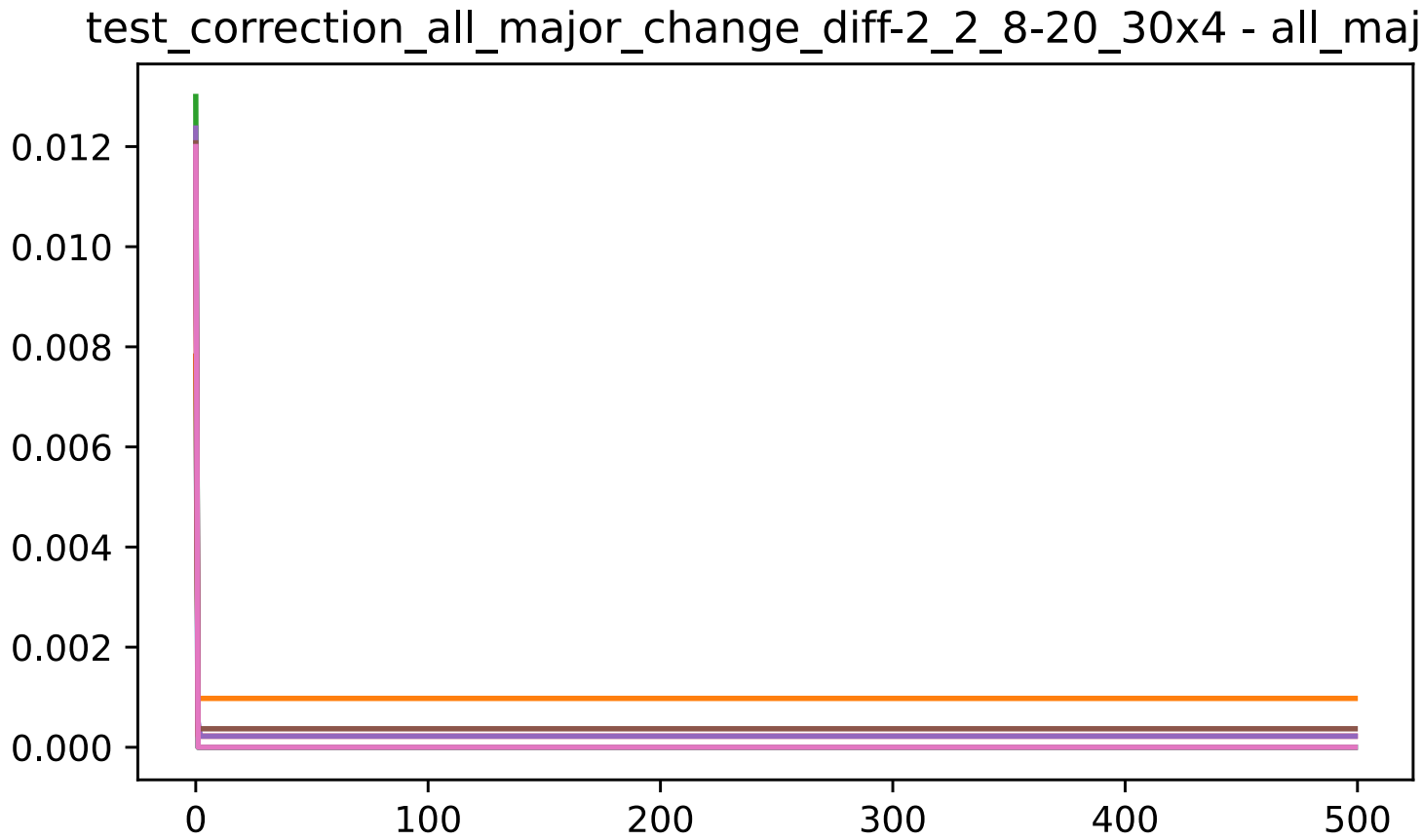
side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

Majority



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

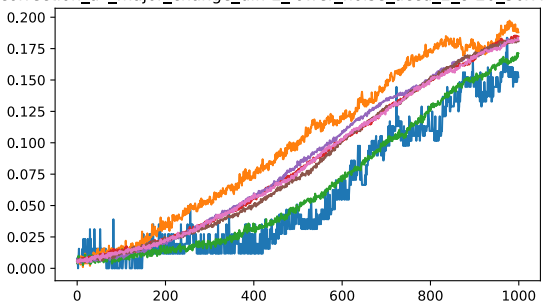
μ-Majority



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

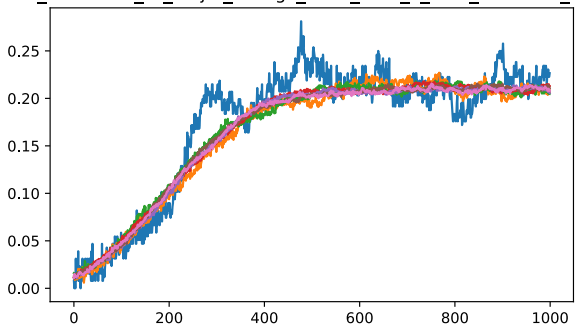
μ-Majority

test_correction_all_major_change_diff-2_lower_noise_accu_2_8-20_30x4 - all_major



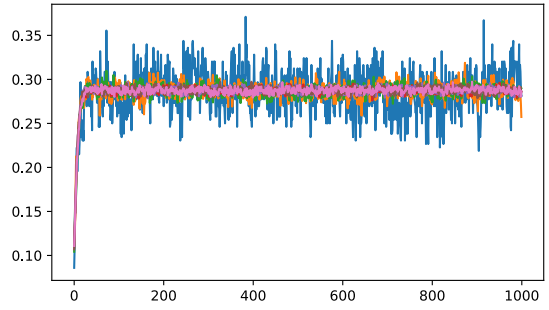
side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.0005]
error accumulation :True

test_correction_all_major_change_diff-2_accu_2_8-20_30x4 - all_major



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

test_correction_all_major_change_diff-2_high_noise_accu_2_8-20_30x4 - all_major



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.01]
error accumulation :True

Colors

Alg.

At the initialization, We divide the bits into subsets (colors). Bits of the same color don't share checks. Then, at ith correction round only bits associated with the ith color [1] apply a correction.

Justification.

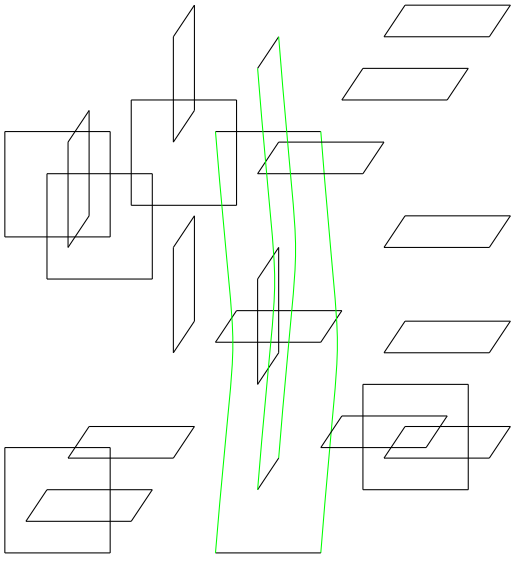
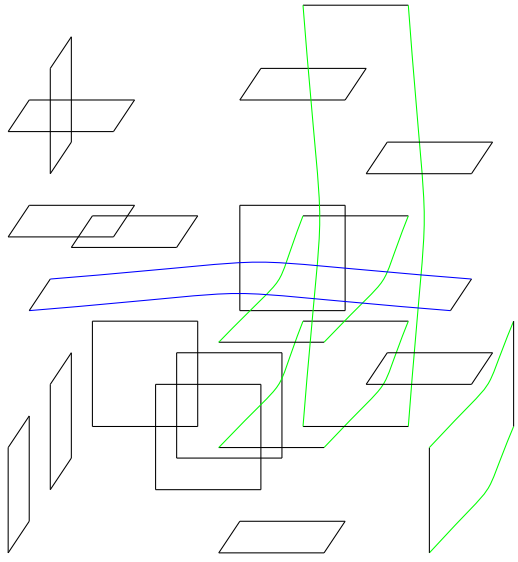
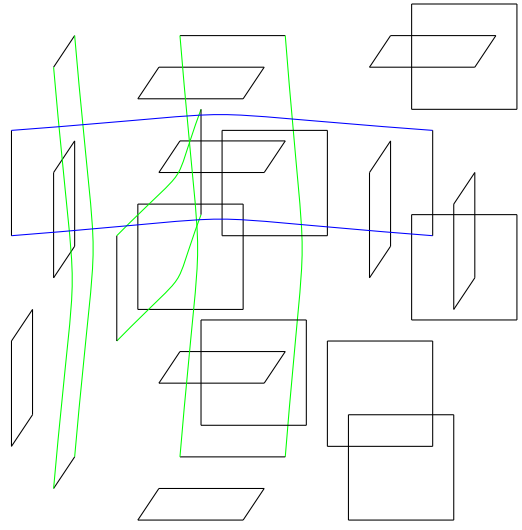
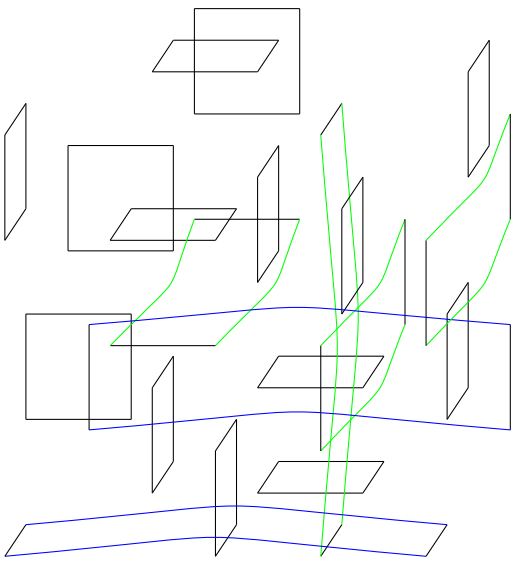
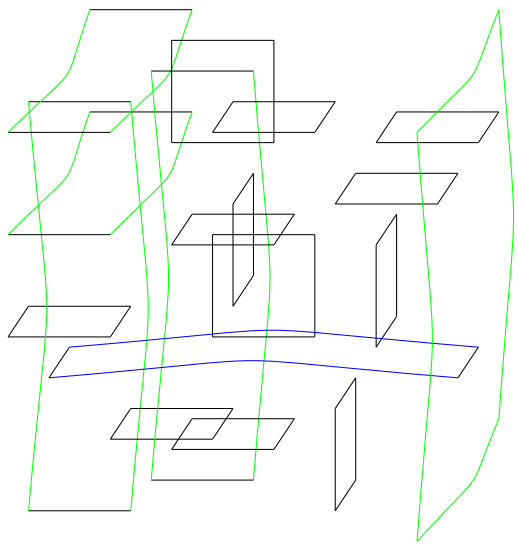
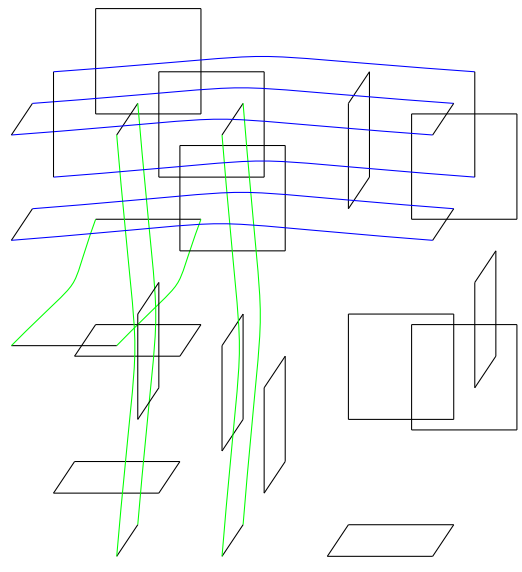
1. No collisions, means that bits don't wait until they can read the syndrome.
2. The independence might help when doing math.

Bottom line.

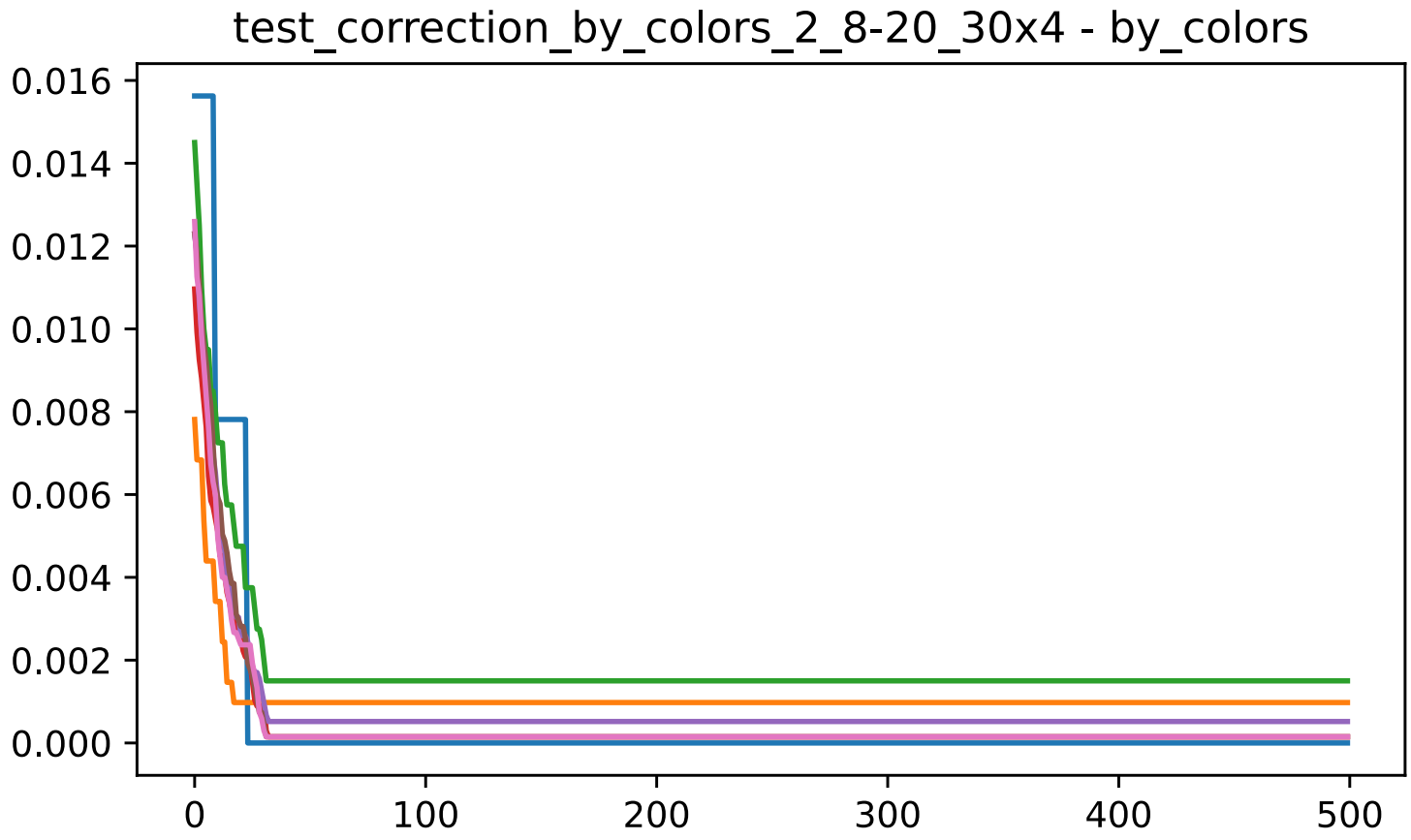
- I took 32 colors and sample a random coloring.

[1] $i \mod \textbf{COLORS}$

Colors

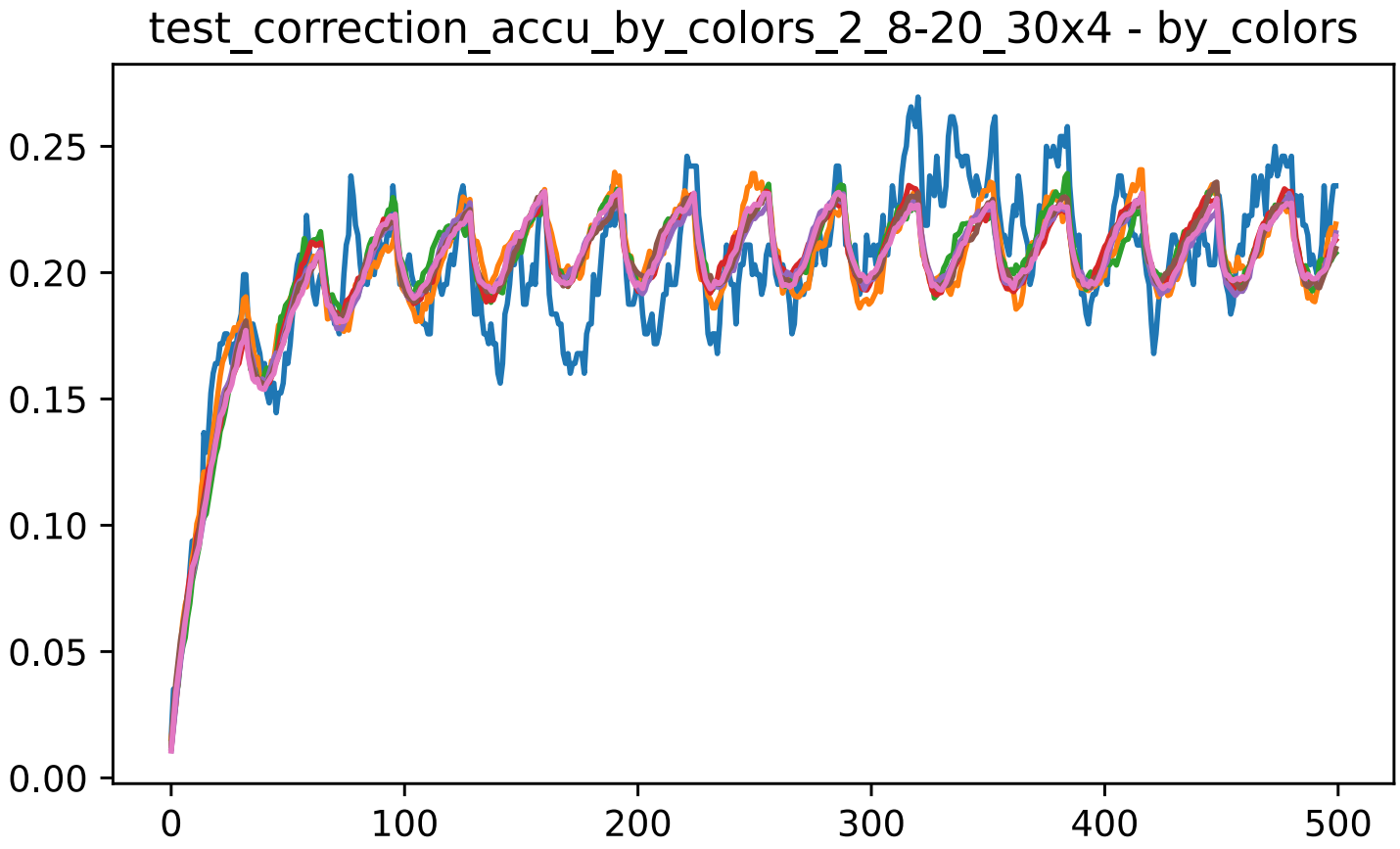







Colors



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

Colors



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

Picking Random Pair

Alg.

Each bit pick two random neighbour checks, and then check if both of them are not satisfied. If so, then flip itself.

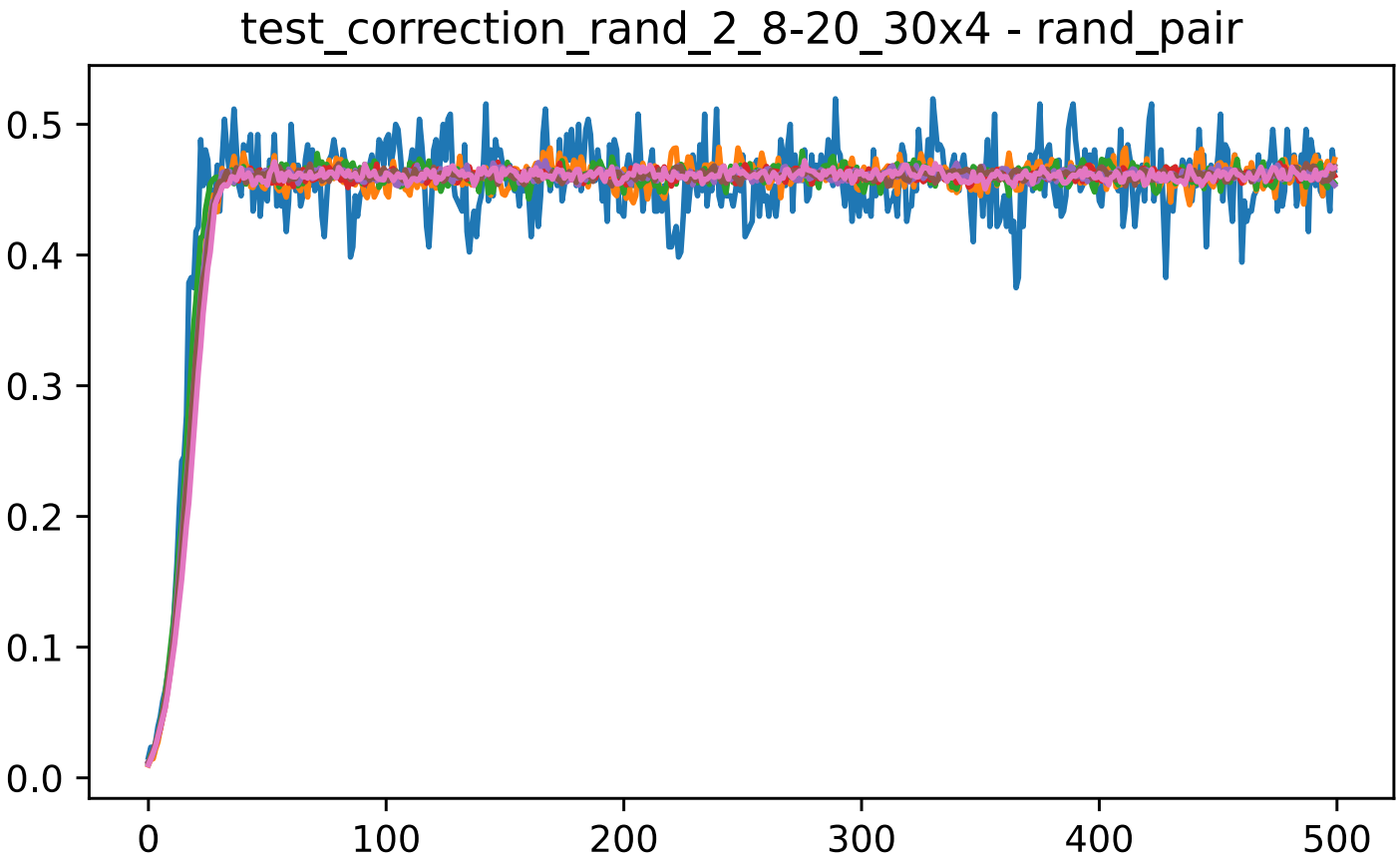
Justification.

With good probability, no too many collisions. Yet, all the bits participate in each round.

Bottom line.

- Fails to correct even in the absence of noise.
- Can we find an analog to $(\frac{1}{2} + \mu)$-majority? Maybe picking 3?

Picking Random



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

Swift Rule

Alg.

We define a north vector $\mathbf{1} = (1, 1, 1..1)$, Then each **vertex** looks only on the **cells** on its neighbourhood such that they are positive relative to $\mathbf{1}$.
Then, the vertex suggest a correction over those cells, that decreases the local syndrome.

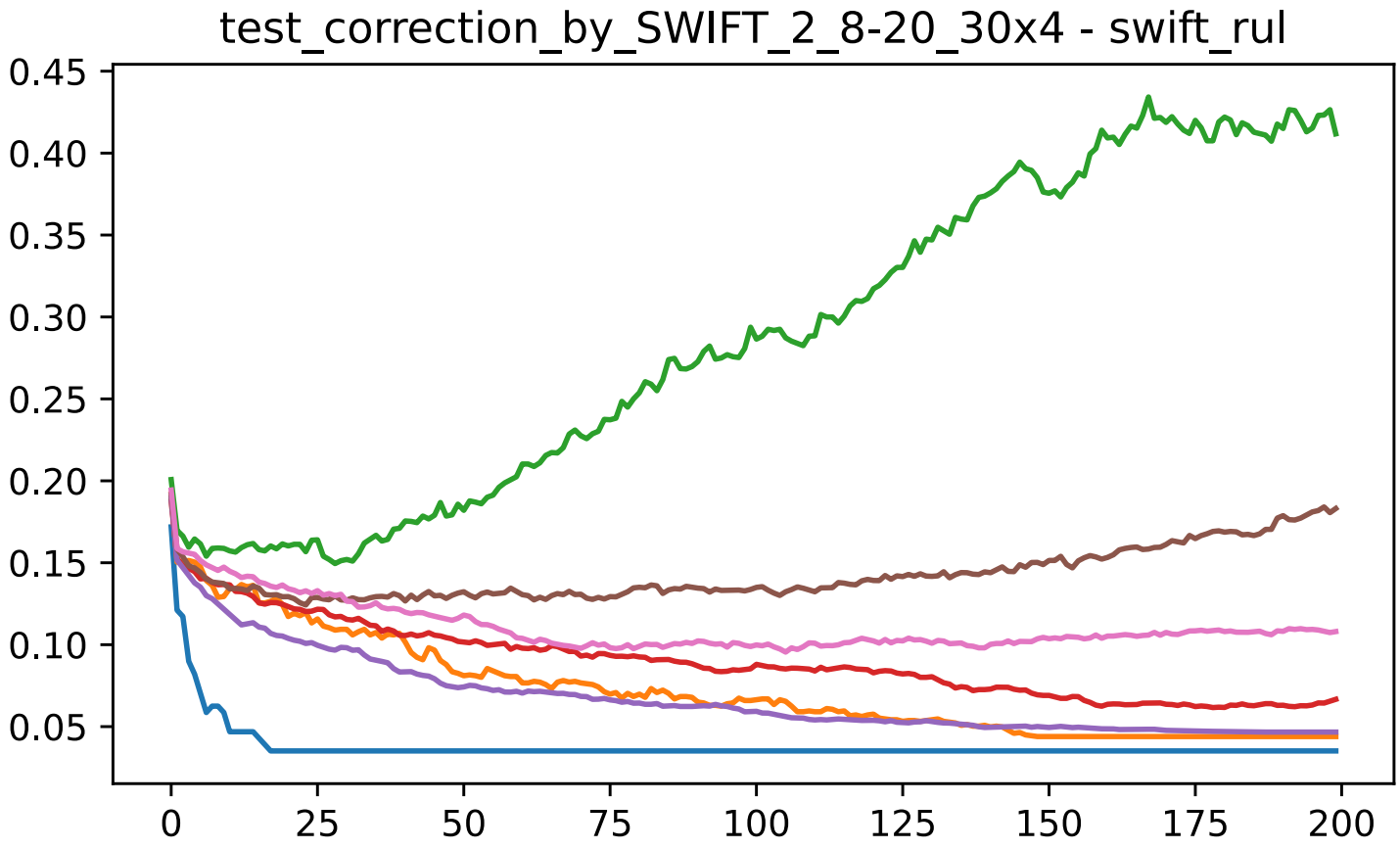
Justification.

1. In the absence of noise, was proven to work.
2. Considered a generalization of Toom's rule.

Bottom line.

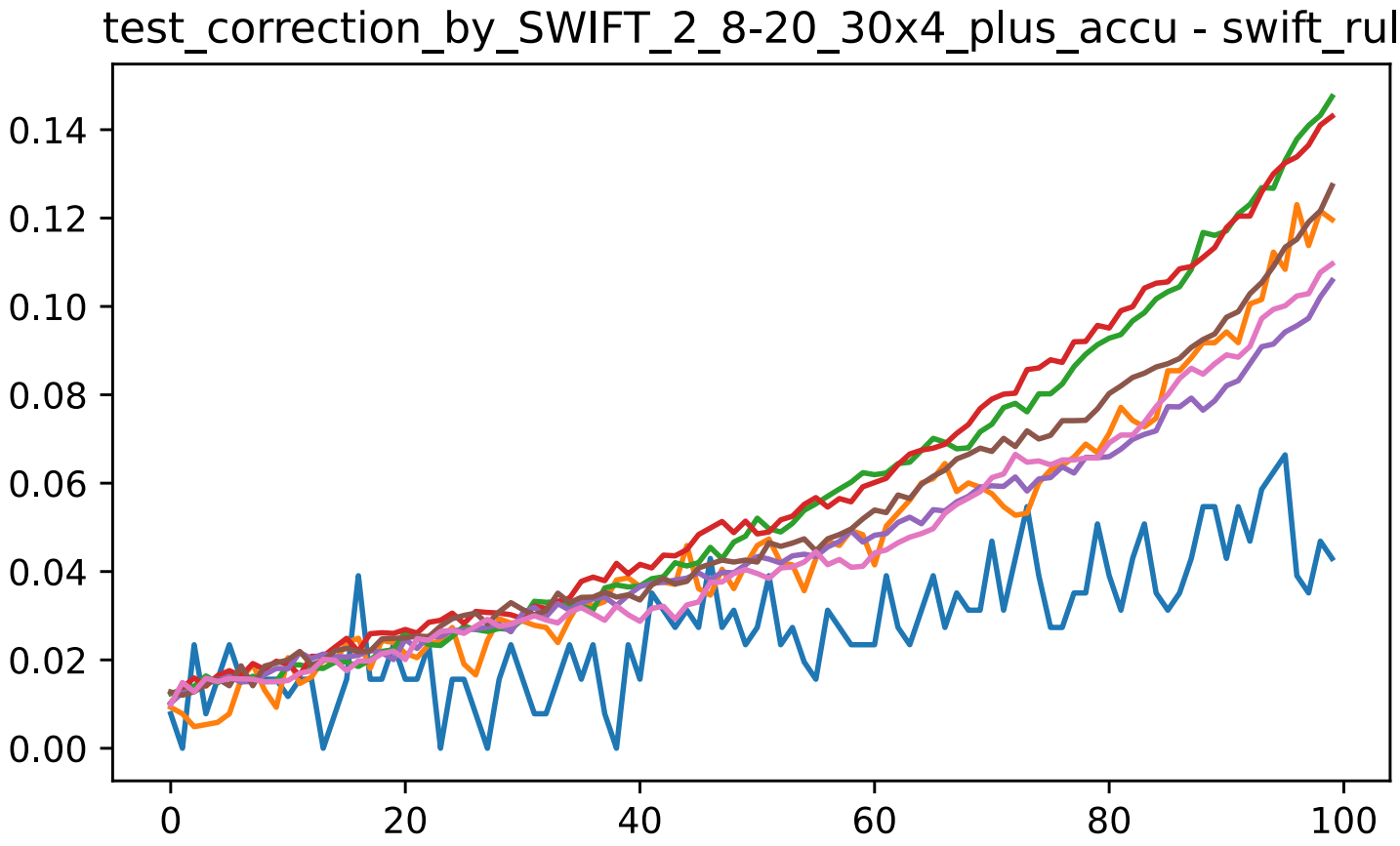
- Works in the absence of noise.

SWIFT



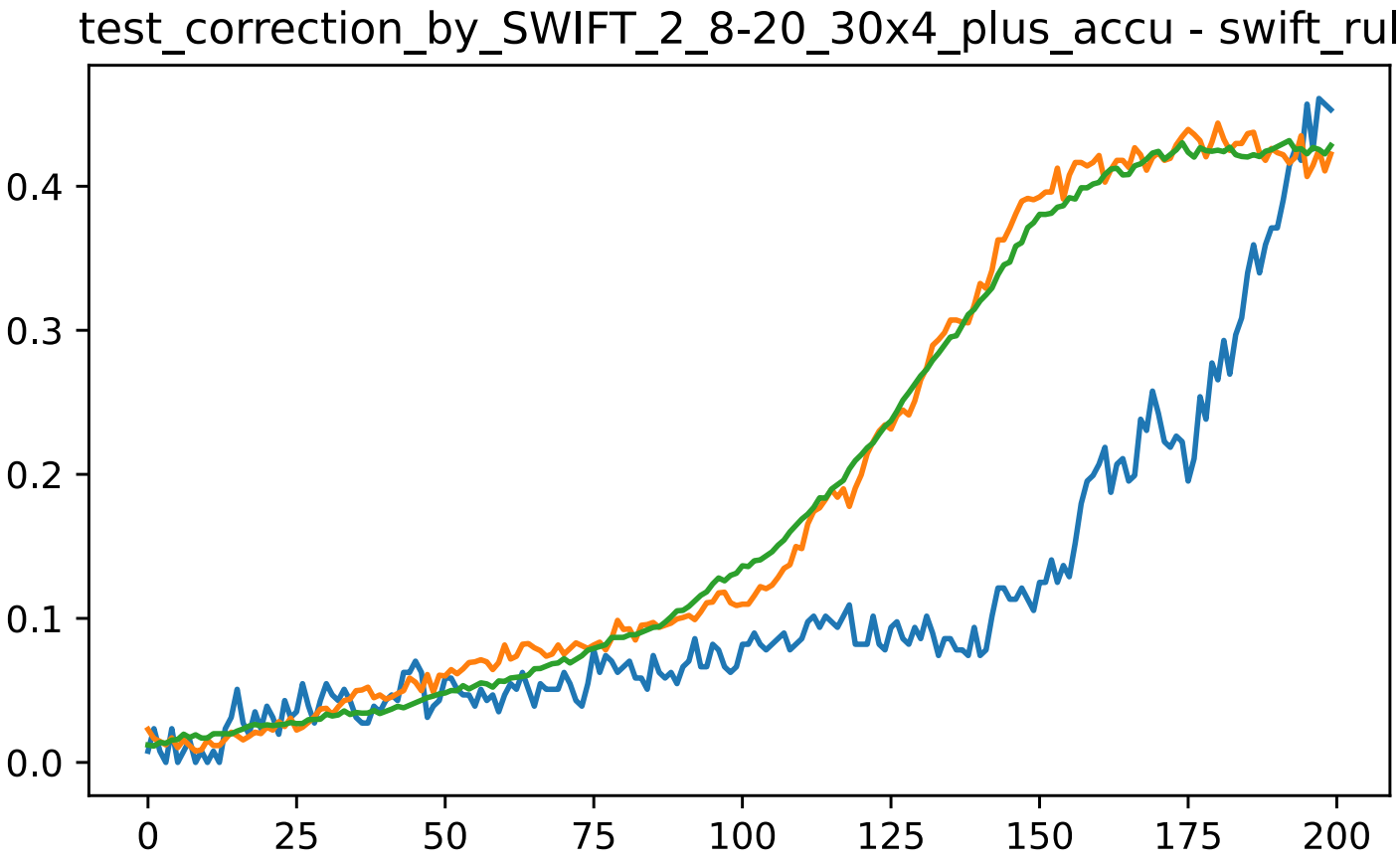
side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.02]
error accumulation :False

SWIFT



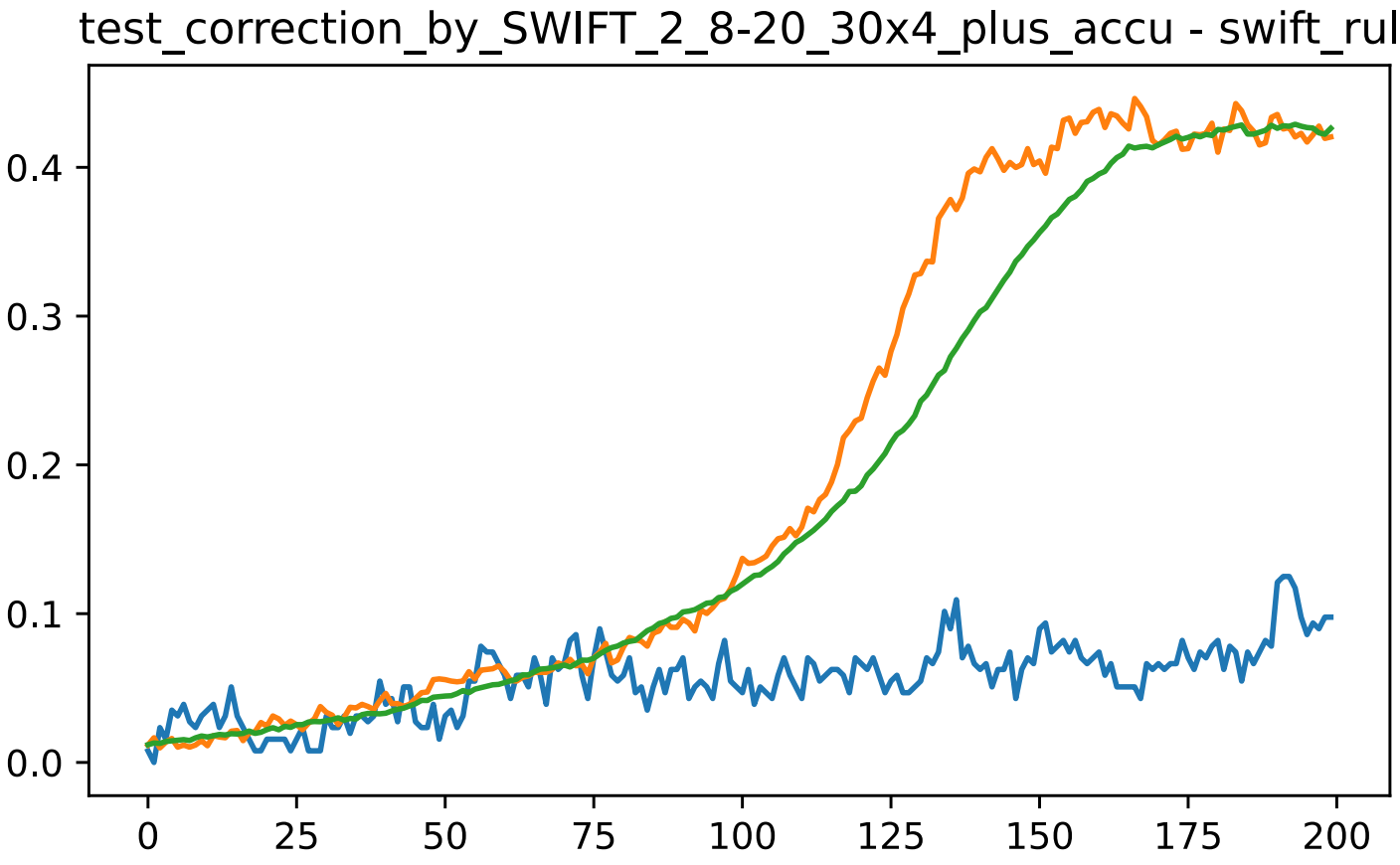
side-length :[8, 16, 30, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

SWIFT



side-length :[8, 16, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

SWIFT



side-length :[8, 16, 40]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

SWIFT

What did we learn?

1. Storing time is, likily, not $\Theta(L)$, might be. $\Theta(L^{\alpha})$ for $\alpha \in (0, 1)$.
2. $\Rightarrow$ implies that the structure of the theorem/key-lemma is diffrernet than known refrigerator.

Bounded Communication Fault Tolerance

Setting

We allow a non-nosy, storng as we wish, classical computation aside. Yet we would like to limit the communication to a constant number of bits per logical qbit, in each round.

Justification

1. If we don't have a constant overhead in depth, in that model, $\Rightarrow$ we also don't have it in the standard model.

Bounded Communication Fault Tolerance

Setting

We allow a non-nosy, storng as we wish, classical computation aside. Yet we would like to limit the communication to a constant number of bits per logical qbit, in each round.

Justification

1. If we don't have a constant overhead in depth, in that model, $\Rightarrow$ we also don't have it in the standard model.
2. In that model, gate teleportation for a constant dimension is easy. Namel, for the Topological codes the task reduced only to find a decoding with constant hint.

Max Color

Alg.

In each round the classical computer, send us the color on which the error is supported the most. Then, each bit correct according majority.

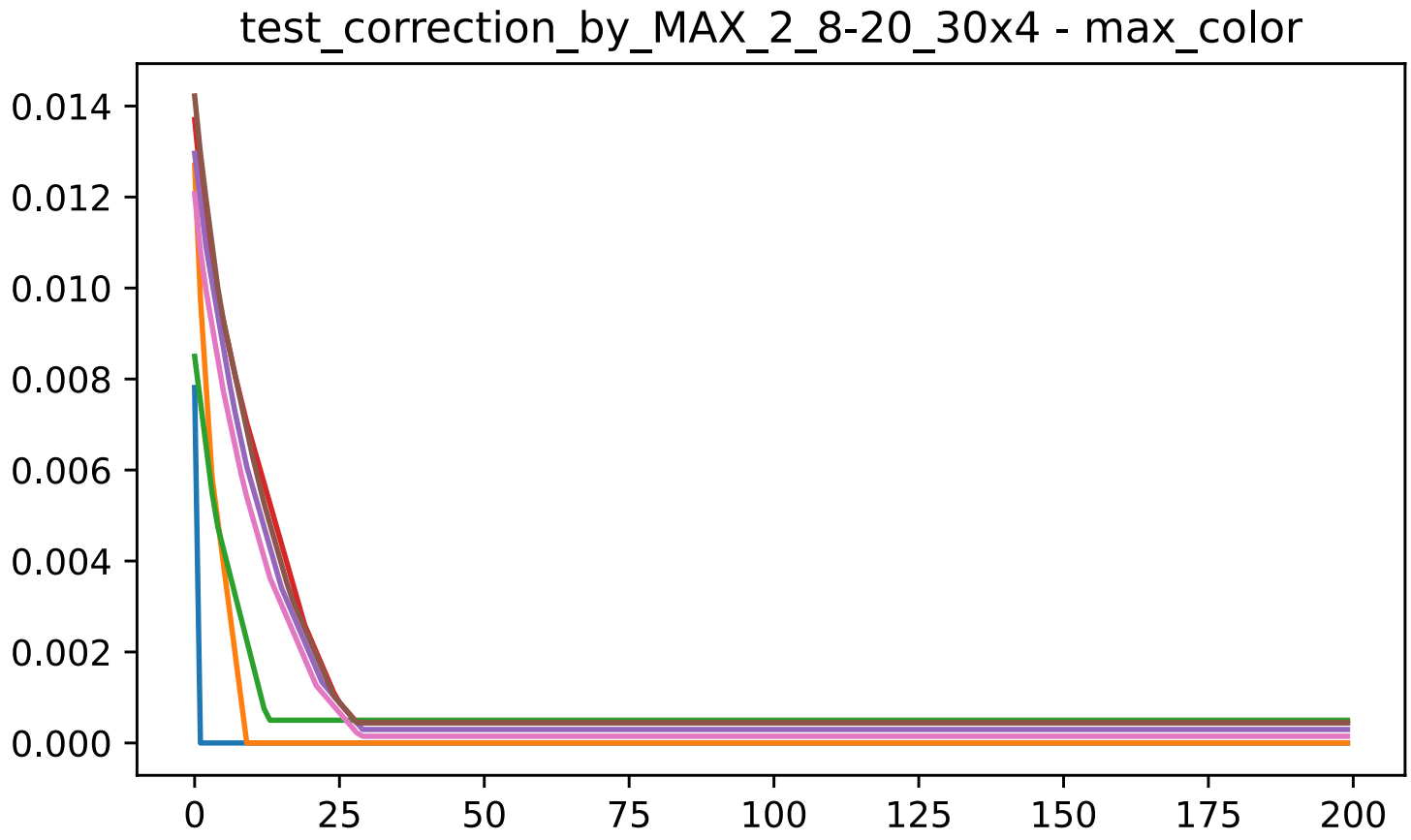
Justification.

1. We don't have works about bounded communication fault tolerance.
2. From mathematical point of view, if the number of colors is c, then we handle a fraction of at least $1/c$ of the dirty bits.

Bottom line.

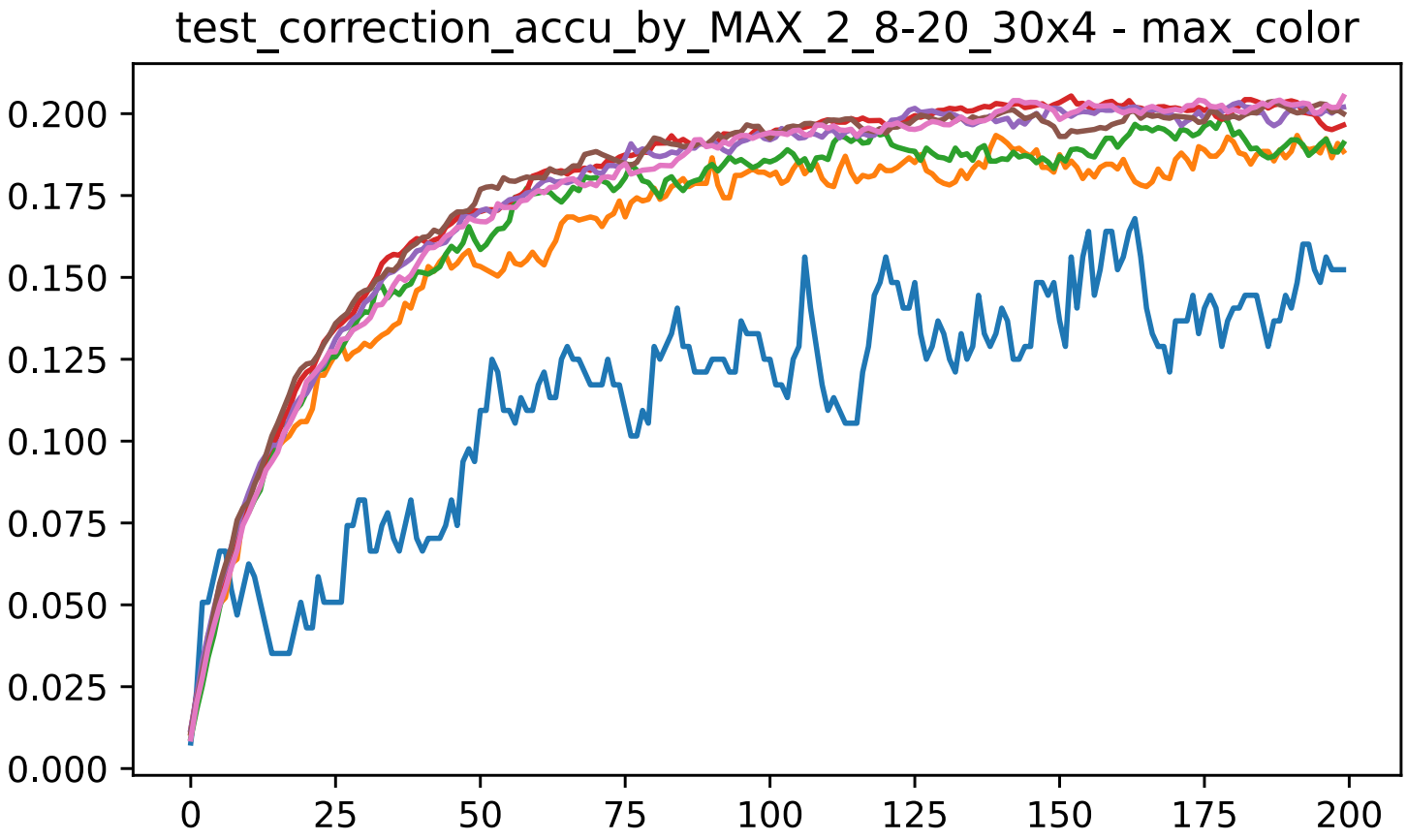
- Works in the absence of noise.

Max Color



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :False

Max Color



side-length :[8, 16, 20, 30, 30, 30, 30]
grid dim :[3, 3, 3, 3, 3, 3, 3]
error rates :[0.001]
error accumulation :True

Max Color

$$\mathbf{Pr}\left[\mathbf{Sup}\left(E_2\right) = S\right] \leq \mathbf{Pr}\left[\text{any bit } v \in S_{c_1} \text{ sees } \mu\text{-majority of satisfied checks }\right] \leq q^{\frac{1}{2}\Delta|S|_{c_1}}$$

Now, to roughly analyze the error after observing a round of p-depolarized noise, we consider a model in which new errors due to the depolarized channel don't correct previous errors. Thus, we get:

$$\mathbf{Pr}\left[\mathbf{Sup}\left(E_3\right) = S\right] \leq \sum_{S' \subset S} q^{\frac{1}{2}\Delta|S'|_{c_1}} p^{|S/S'|}$$

On the other hand,

$$\sum_{c_i} |S|_{c_i} = k \cdot \mathbf{E}\left[|S|_{c_i}\right] = |S|$$

$$\Rightarrow \max_{c_i} |S|_{c_i} \geq \frac{1}{k}|S|$$

So if c_1 is the color that maximizes $|S|_{c_1}$, then:

$$\begin{aligned}\mathbf{Pr}\left[\mathbf{Sup}\left(E_3\right) = S\right] &\leq \sum_{S' \subset S} q^{\frac{1}{2}\Delta|S'|/k} p^{|S/S'|} \\ &\leq \left(q^{\frac{1}{2k}\Delta} + p\right)^{|S|} \leq q^{|S|}\end{aligned}$$

$$C_{\frac{1}{2}-\mu}^{\frac{1}{2}} \left(q^{\left((\Delta_2 - 1)\log_q(2)+1\right)\left(\frac{1}{2}-\mu\right)+\log_q 2}\right)^{|Q|}$$

Future

So What's Next?

1. Moving forward with the experimental work?
   - More experiments, higher statistical standards?
   - Accelerate the decoder? Shifting to $c/c++$? Parallelize it with GPUs? Do we have budget?

Future

So What's Next?

1. Moving forward with the experimental work?
   - More experiments, higher statistical standards?
   - Accelerate the decoder? Shifting to $c/c++$? Parallelize it with GPUs? Do we have budget?
2. Dividing to colors - Expander Decomposition? What do we know about the expander decomposition of the periodic grid?

Future

So What's Next?

1. Moving forward with the experimental work?
   - More experiments, higher statistical standards?
   - Accelerate the decoder? Shifting to $c/c++$? Parallelize it with GPUs? Do we have budget?
2. Dividing to colors - Expander Decomposition? What do we know about the expander decomposition of the periodic grid?
3. How to use the fact our code is a quotient code?